Pertemuan 8

Moda Self-Study: Implementasi Tree pada Game

Disusun Untuk Memenuhi Tugas Mata Kuliah Praktikum Terapan Teori Graf

Dosen Pengampu:

TIM DOSEN

DISUSUN OLEH:

MUHAMMAD RAFLY ROMEO NASUTION

10123875

3KA25

FAKULTAS ILMU KOMPUTER DAN TEKNOLOGI INFORMASI

SISTEM INFORMASI

UNIVERSITAS GUNADARMA

ATA 2025/2026

Analisis Program Prolog: Family Knowledge Base

1. FAKTA (FACTS)

Fakta adalah pernyataan yang dianggap benar tanpa syarat apapun. Prolog langsung menerima fakta sebagai kebenaran mutlak.

1.1. Fakta Jenis Kelamin

```
male(lennart).
male(mike).
male(donald).
male(usain).
male(joar).
male(adam).
male(dan).
male(simon).
female(hillary).
female(elise).
female(lisa).
female(lena).
```

Penjelasan logika:

Prolog menyimpan setiap fakta ini di dalam database-nya. Ketika ada query yang menanyakan apakah seseorang laki-laki atau perempuan, Prolog hanya mencocokkan nama tersebut dengan daftar fakta yang ada. Jika ditemukan, hasilnya true. Jika tidak ditemukan, hasilnya false.

Contoh cara Prolog memproses query **male(donald)**:

```
Prolog memeriksa database dari atas ke bawah:

- male(lennart) ⟶ "lennart" cocok dengan "donald"? TIDAK

- male(mike)    ⟶ "mike" cocok dengan "donald"?    TIDAK

- male(donald)  ⟶ "donald" cocok dengan "donald"?  YA ⟶ true
```

1.2. Fakta Relasi Orang Tua

```
parent(mike, lennart).
parent(mike, lena).
parent(lennart, donald).
parent(lennart, hillary).
parent(lennart, usain).
parent(lena, adam).
parent(lena, simon).
parent(adam, dan).
parent(donald, lisa).
parent(hillary, joar).
parent(hillary, elise).
```

Penjelasan logika:

Format parent(X, Y) berarti X adalah orang tua dari Y. Prolog membaca ini sebagai relasi dua arah antara dua entitas. Ketika ada query parent(lennart, donald), Prolog menelusuri seluruh daftar fakta parent dari atas ke bawah sampai menemukan yang cocok.

Cara Prolog memproses query **parent(lennart, donald)**:

```
- parent(mike, lennart)    ⟶ argumen pertama "mike" cocok "lennart"? TIDAK

- parent(mike, lena)       ⟶ argumen pertama "mike" cocok "lennart"? TIDAK

- parent(lennart, donald)  ⟶ "lennart" cocok "lennart"? YA

                              "donald" cocok "donald"?   YA ⟶ true
```

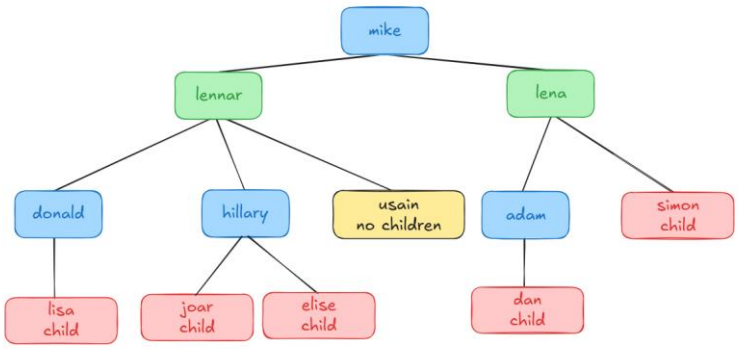
1.3. Fakta Anak Generasi Terakhir

```
child(lisa).
child(joar).
child(elise).
child(dan).
child(simon).
```

Penjelasan logika:

Fakta child digunakan sebagai penanda bahwa seseorang adalah daun (leaf node) dari pohon keluarga, artinya mereka tidak memiliki anak dalam database ini. Fakta ini menjadi base case untuk aturan rekursif family_children yang akan dijelaskan nanti. Tanpa fakta ini, rekursi tidak tahu kapan harus berhenti.

1.4. Struktur Pohon Keluarga Berdasarkan Fakta\



2. ATURAN (RULES)

Aturan adalah logika yang dibangun dari fakta-fakta yang sudah ada. Format aturan di Prolog adalah:

```
kepala(X, Y) :- syarat1(X), syarat2(Y).
```

Dibaca: "kepala berlaku jika syarat1 DAN syarat2 terpenuhi". Tanda :- berarti "jika", dan tanda koma , berarti "dan".

2.1. Aturan father

```
father(X,Y) :- male(X),parent(X,Y).
```

Penjelasan logika:

X adalah ayah dari Y apabila dua syarat terpenuhi secara bersamaan. Pertama, X harus terdaftar sebagai laki-laki di fakta male. Kedua, X harus terdaftar sebagai orang tua dari Y di fakta parent. Prolog memeriksa syarat pertama dulu. Jika syarat pertama gagal, syarat kedua tidak perlu diperiksa karena hasilnya sudah pasti gagal.

Cara kerja Prolog saat menerima query **father(X, donald)**:

```
Prolog mencari nilai X yang membuat kedua syarat terpenuhi.
Coba X = lennart:
  Syarat 1: male(lennart) → ada di database? YA, lanjut
  Syarat 2: parent(lennart,donald) → ada di database? YA
Kedua syarat terpenuhi → X = lennart BERHASIL
Prolog berhenti karena sudah menemukan hasil pertama. Jika
user menekan titik koma untuk mencari hasil lain, Prolog akan
melanjutkan pencarian dari titik terakhir.
```

2.2. Aturan mother

```
mother(X,Y) :- female(X),parent(X,Y).
```

Penjelasan logika:

X adalah ibu dari Y apabila X terdaftar sebagai perempuan di fakta female, dan X terdaftar sebagai orang tua dari Y di fakta parent. Mekanisme kerjanya identik dengan aturan father, perbedaannya hanya pada syarat pertama yang menggunakan female bukan male.

Cara kerja Prolog saat menerima query **mother(X, joar)**:

```
Prolog mencari nilai X yang membuat kedua syarat terpenuhi.
Coba X = hillary:
  Syarat 1: female(hillary) -> ada di database? YA, lanjut
  Syarat 2: parent(hillary, joar) -> ada di database? YA
  Kedua syarat terpenuhi -> X = hillary BERHASIL
```

2.3. Aturan son

```
son(X,Y) :- male(X),parent(Y,X).
```

Penjelasan logika:

X adalah anak laki-laki dari Y apabila X terdaftar sebagai laki-laki, dan Y terdaftar sebagai orang tua dari X. Perlu diperhatikan bahwa posisi argumen di parent ditulis parent(Y, X) bukan parent(X, Y). Ini karena dalam relasi ini yang menjadi orang tua adalah Y, bukan X.

Cara kerja Prolog saat menerima query **son(X, mike)**:

```
Prolog mencoba semua nilai X dari daftar male satu per satu.

Coba X = lennart:
  Syarat 1: male(lennart)         -> YA, lanjut
  Syarat 2: parent(mike, lennart) -> YA
  Berhasil -> X = lennart

Coba X = mike (jika user minta hasil berikutnya):
  Syarat 1: male(mike)            -> YA, lanjut
  Syarat 2: parent(mike, mike)    -> TIDAK ADA di database -> GAGAL

Coba X = donald:
  Syarat 1: male(donald)          -> YA, lanjut
  Syarat 2: parent(mike, donald)  -> TIDAK ADA
            (orang tua donald adalah lennart, bukan mike) -> GAGAL

Coba X = usain:
  Syarat 1: male(usain)           -> YA, lanjut
  Syarat 2: parent(mike, usain)   -> TIDAK ADA -> GAGAL

(Prolog terus mencoba sisa daftar male, semuanya gagal)
Hasil akhir hanya satu: X = lennart
```

2.4. Aturan daughter

```
daughter(X,Y) :- female(X),parent(Y,X).
```

Penjelasan logika:

X adalah anak perempuan dari Y apabila X terdaftar sebagai perempuan, dan Y terdaftar sebagai orang tua dari X. Sama seperti son, posisi argumen di parent dibalik karena yang menjadi orang tua adalah Y bukan X.

Cara kerja Prolog saat menerima query **daughter(X, lennart)**:

```
Prolog mencoba semua nilai X dari daftar female satu per satu.

Coba X = hillary:
  Syarat 1: female(hillary)          -> YA, lanjut
  Syarat 2: parent(lennart, hillary) -> YA
  Berhasil -> X = hillary

Coba X = elise (jika user minta hasil berikutnya):
  Syarat 1: female(elise)            -> YA, lanjut
  Syarat 2: parent(lennart, elise)   -> TIDAK ADA
            (orang tua elise adalah hillary) -> GAGAL

Coba X = lisa:
  Syarat 1: female(lisa)             -> YA, lanjut
  Syarat 2: parent(lennart, lisa)    -> TIDAK ADA
            (orang tua lisa adalah donald) -> GAGAL

Coba X = lena:
  Syarat 1: female(lena)             -> YA, lanjut
  Syarat 2: parent(lennart, lena)    -> TIDAK ADA
            (orang tua lena adalah mike) -> GAGAL

Hasil akhir hanya satu: X = hillary
```

2.5. Aturan family_children (Rekursif)

```
family_children(X, X):-
    child(X).

family_children(X, Child):-
    parent(X,Y),
    family_children(Y, Child).
```

Penjelasan logika:

Aturan ini terdiri dari dua klausa yang bekerja bersama untuk menelusuri seluruh keturunan seseorang sampai ke generasi paling bawah.

Klausa 1 - Base Case:

```
family_children(X, X) :- child(X).
```

X adalah keturunan dari dirinya sendiri jika X terdaftar sebagai child. Ini adalah titik berhenti dari rekursi. Ketika Prolog mencapai seseorang yang terdaftar sebagai child, penelusuran berhenti dan nama tersebut dikembalikan sebagai hasil akhir.

Klausa 2 - Recursive Case:

```
family_children(X, Child) :- parent(X, Y),
family_children(Y, Child).
```

Child adalah keturunan dari X jika ada Y yang merupakan anak langsung dari X, dan Child adalah keturunan dari Y. Prolog memanggil dirinya sendiri dengan Y sebagai titik awal baru, terus menelusuri ke bawah hingga base case terpenuhi.

Cara kerja Prolog untuk query **family_children(mike, X)**:

```
family_children(mike, X)
|
+-- Klausa 1: child(mike) -> TIDAK ADA -> GAGAL
|
+-- Klausa 2: parent(mike, Y)
    |
    +-- Y = lennart -> family_children(lennart, X)
    |   |
    |   +-- Klausa 1: child(lennart) -> TIDAK ADA -> GAGAL
    |   |
    |   +-- Klausa 2: parent(lennart, Y)
    |       |
    |       +-- Y = donald -> family_children(donald, X)
    |       |   |
    |       |   +-- Klausa 1: child(donald) -> TIDAK ADA ->
GAGAL
    |       |   |
    |       |   +-- Klausa 2: parent(donald, Y)
    |       |       |
    |       |       +-- Y = lisa -> family_children(lisa, X)
    |       |           |
    |       |           +-- Klausa 1: child(lisa) -> YA
    |       |               X = lisa -> [HASIL 1]
    |       |
    |       +-- Y = hillary -> family_children(hillary, X)
    |       |   |
    |       |   +-- Klausa 1: child(hillary) -> TIDAK ADA -
> GAGAL
    |       |   |
    |       |   +-- Klausa 2: parent(hillary, Y)
    |       |       |
    |       |       +-- Y = joar -> family_children(joar, X)
    |       |       |   |
    |       |       |   +-- Klausa 1: child(joar) -> YA
    |       |       |       X = joar -> [HASIL 2]
    |       |       |
```

```
    |       |       +-- Y = elise -> family_children(elise,
X)
    |       |           |
    |       |           +-- Klausa 1: child(elise) -> YA
    |       |               X = elise -> [HASIL 3]
    |       |
    |       +-- Y = usain -> family_children(usain, X)
    |           |
    |           +-- Klausa 1: child(usain)     -> TIDAK ADA
-> GAGAL
    |           +-- Klausa 2: parent(usain, Y) -> TIDAK ADA
-> GAGAL
    |           [TIDAK ADA HASIL]
    |
    +-- Y = lena -> family_children(lena, X)
        |
        +-- Klausa 1: child(lena) -> TIDAK ADA -> GAGAL
        |
        +-- Klausa 2: parent(lena, Y)
            |
            +-- Y = adam -> family_children(adam, X)
            |   |
            |    +-- Klausa 1: child(adam) -> TIDAK ADA ->
GAGAL
            |   |
            |   +-- Klausa 2: parent(adam, Y)
            |       |
            |       +-- Y = dan -> family_children(dan, X)
            |           |
            |           +-- Klausa 1: child(dan) -> YA
            |               X = dan -> [HASIL 4]
            |
            +-- Y = simon -> family_children(simon, X)
                |
                +-- Klausa 1: child(simon) -> YA
                    X = simon -> [HASIL 5]
```

3. OUTPUT (QUERY)

Query adalah pertanyaan yang diajukan ke Prolog untuk mencari nilai yang memenuhi kondisi tertentu. Prolog menjawab dengan menelusuri fakta dan aturan yang ada, lalu mengembalikan semua nilai yang memenuhi syarat.

Query 1: Siapa ayah dari donald?

```
?- father(X, donald).
```

Prolog memeriksa aturan father(X,Y) :- male(X), parent(X,Y) dengan Y = donald:

```
Coba X = lennart:
  male(lennart)          -> YA
  parent(lennart,donald) -> YA
  -> BERHASIL
```

Output:

```
24 ?- father(X, donald).
X = lennart .
```

Query 2: Siapa ibu dari joar?

```
?- mother(X, joar).
```

Prolog memeriksa aturan mother(X,Y) :- female(X), parent(X,Y) dengan Y = joar:

```
Coba X = hillary:
  female(hillary)       -> YA
  parent(hillary, joar) -> YA
  -> BERHASIL
```

Output:

```
25 ?- mother(X, joar).
X = hillary .
```

Query 3: Siapa saja anak laki-laki dari mike?

```
?- son(X, mike).
```

Prolog memeriksa aturan son(X,Y) :- male(X), parent(Y,X) dengan Y = mike:

```
Coba X = lennart:
  male(lennart)          -> YA
  parent(mike, lennart) -> YA
  -> BERHASIL

Coba X = mike:
  male(mike)             -> YA
  parent(mike, mike)     -> TIDAK ADA -> GAGAL

Coba X = donald:
  male(donald)           -> YA
  parent(mike, donald)   -> TIDAK ADA -> GAGAL

Coba X = usain:
  male(usain)            -> YA
  parent(mike, usain)    -> TIDAK ADA -> GAGAL

Coba X = joar, adam, dan, simon -> semuanya GAGAL
Tidak ada parent(mike, ...) yang cocok selain lennart.
```

Output:

```
26 ?- son(X, mike).
X = lennart .
```

Query 4: Siapa saja anak perempuan dari lennart?

```
?- daughter(X, lennart).
```

Prolog memeriksa aturan daughter(X,Y) :- female(X), parent(Y,X) dengan Y = lennart:

```
Coba X = hillary:
  female(hillary)          -> YA
  parent(lennart, hillary) -> YA
  -> BERHASIL

Coba X = elise:
  female(elise)            -> YA
  parent(lennart, elise)   -> TIDAK ADA -> GAGAL

Coba X = lisa:
  female(lisa)             -> YA
  parent(lennart, lisa)    -> TIDAK ADA -> GAGAL

Coba X = lena:
  female(lena)             -> YA
  parent(lennart, lena)    -> TIDAK ADA -> GAGAL
```

Output:

```
27 ?- daughter(X, lennart).
X = hillary .
```

Query 5: Siapa saja keturunan dari mike?

```
?- family_children(mike, X).
```

Prolog menelusuri pohon keluarga secara rekursif dari mike ke bawah. Setiap cabang ditelusuri sampai menemukan node yang terdaftar sebagai child:

```
mike -> lennart -> donald -> lisa    (child) -> HASIL 1
mike -> lennart -> hillary -> joar   (child) -> HASIL 2
mike -> lennart -> hillary -> elise  (child) -> HASIL 3
mike -> lennart -> usain             (bukan child, tidak punya anak) -> GAGAL
mike -> lena    -> adam   -> dan     (child) -> HASIL 4
mike -> lena    -> simon             (child) -> HASIL 5
```

Output:

```
28 ?- family_children(mike, X).
X = lisa ;
X = joar ;
X = elise ;
X = dan ;
X = simon .
```

Query 6: Apakah usain punya keturunan?

```
?- family_children(usain, X).
```

Prolog mencoba kedua klausa untuk usain:

```
Klausa 1: child(usain) -> TIDAK ADA di database -> GAGAL
Klausa 2: parent(usain, Y) -> TIDAK ADA di database -> GAGAL

Semua jalur gagal, tidak ada hasil yang bisa dikembalikan.
```

Output:

```
29 ?- family_children(usain, X).
false.
```

Query 7: Siapa saja keturunan dari lennart?

```
?- family_children(lennart, X).
```

Prolog menelusuri pohon dari lennart ke bawah:

```
lennart -> donald  -> lisa   (child) -> HASIL 1
lennart -> hillary -> joar   (child) -> HASIL 2
lennart -> hillary -> elise  (child) -> HASIL 3
lennart -> usain             (bukan child, tidak punya anak)
-> GAGAL
```

Output:

```
31 ?- family_children(lennart, X).
X = lisa ;
X = joar ;
X = elise .
```